Technical Report

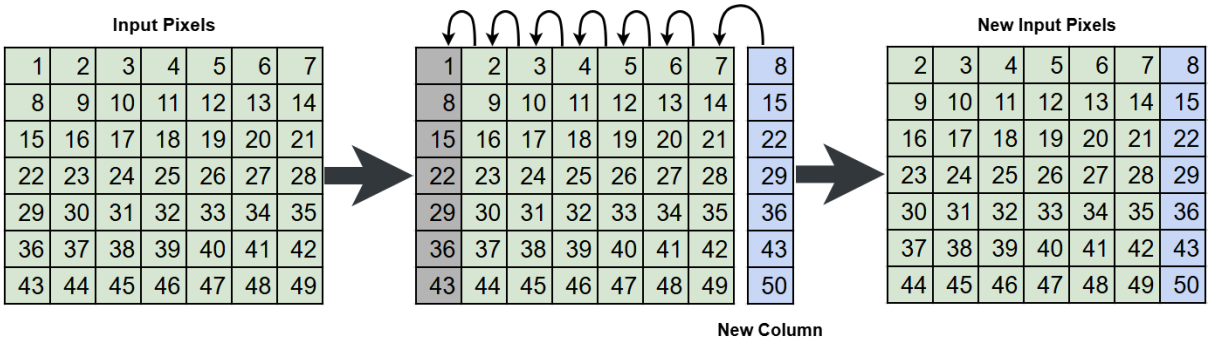
1. Introduction

The objective of this project was to design and implement a custom IP block for 2D convolution filter on the Zynq-7000 SoC using VHDL and C. The core component is a custom AXI4-Lite IP block capable of performing spatial filtering with a 7x7 sliding window. By offloading intensive pixel calculations from the ARM processor to Programmable Logic (PL), the system leverages parallel processing to achieve significant performance improvements over software-only solutions.

2. Hardware Architecture & Design Choices

The system integrates the Zynq Processing System (PS) with the custom Filter IP via the AXI Interconnect. The design uses the AXI4-Lite protocol, where the PL acts as a memory-mapped slave. This allows the PS processor to write pixel data and configuration parameters directly to hardware registers.

Design Choice 1: The 7x7 Sliding Window & Column Updating



A standard convolution with a 7x7 kernel requires 49 pixel values for every output calculation. Sending all 49 values for each pixel would create a massive bandwidth bottleneck on the AXI bus. To address this, a Column Update strategy was implemented. The hardware core maintains an internal 2D shift register array. The software sends only the new column (7 pixels) entering the window. The hardware then simultaneously shifts all existing columns to the left and inserts the new data. This architectural choice reduces traffic by a factor of 7 (from 49 writes to 7 writes per pixel), significantly improving throughput.

Design Choice 2: Arithmetic Precision (Signed vs. Unsigned)

Data Element	Type	Range
Input Pixel	Unsigned 8	0 to 255
Coefficient	Signed 16	-32,768 to 32768
Internal Sum	Signed 32	-2.1 Billion to 2.1 Billion
Final Output	Unsigned 8	0 to 255

The design uses unsigned 8-bit logic for pixel data (0-255) to match standard image formats as input image is a gray scale pixel value 0-255. However, for kernel coefficients, signed 16-bit precision (-32,768 to +32,767) was chosen. This distinction is crucial because while pixels are positive, many essential filters like Edge Detection rely on negative weights (e.g., -1). Using signed 16-bit ensures the hardware can implement negative filters without underflow errors. The internal Multiply-Accumulate unit uses a 32-bit signed accumulator to prevent overflow during summation.

Design Choice 3: The Approximation Divider

$$\frac{A}{B} = (A \times M) >> S$$

$$where,\ M = \frac{2^S}{B}$$

True hardware division is resource-heavy and slow. To optimize for speed, a Multiplier-Shift strategy was implemented. Instead of calculating A / B, the hardware calculates (A x M) >> S. The driver software pre-calculates the multiplier M and shift S. By using this method, the system avoids the need for a complex hardware divider, making the mathematical processing much faster and more efficient within the programmable logic.

For example, instead of dividing 100 by 9 (100/9 = 11.11). This method will calculate the multiplier M and Shift S. By fixing S equal to 16, multiplier M will be

$$M = \frac{2^{16}}{9} \approx 7282$$

$$So\ that,\ \frac{100}{9} = (100 \times 7282) >> 16 = 11$$

Although this approach loses the decimal values, the final outputs of an image filter must be integers for pixel display. Consequently, ignoring the decimals allows for a significantly more efficient implementation for real-time hardware processing.

Design Choice 4: Register Map Optimization

Register	Function	Value			
		8 bits [31:24]	8 bits [23:16]	8 bits [15:8]	8 bits [7:0]
0	CONTROL_REG	Multiplier		Shift	Clamp/ABS/EN
2	DATA_IN1	pixel 3	pixel 2	pixel 1	pixel 0
3	DATA_IN2		pixel 6	pixel 5	pixel 4
5	COEFF_0	(signed 16) COEFF_1		(signed 16) COEFF_0	
6	COEFF_2	(signed 16) COEFF_3		(signed 16) COEFF_2	

There are 30 registers in the IP interface, including control, input, output, and coefficient registers. The control register has all of the parameters for the multiplier, shift, output clamp, and absolute output packed in one 32-bit register. For Input register, seven of 8-bit input pixels are packed into two 32-bit registers. Finally, Two signed 16-bit values are packed into a single 32-bit register, with higher byte for the odd coefficient and the lower byte for even coefficient. So, 49 kernels can fit inside 25 of the 32-bit registers, from register 5 to register 29. This technique will reduce the number of register use significantly.

3. Software Driver Implementation

The software stack uses a layered architecture to reduce hardware complexity. Driver Architecture (myfilter_utils.h) The low-level driver provides bit-level operations and some essential functions for Image processing.

- **load_kernel_7x7:** Accepts a normal 2D 7x7 C array as input, and does the bit-packing logic (shifting even indices to lower 16 bits, odd to upper 16 bits) before writing to registers.
- **set_filter_control:** Constructs the 32-bit control word by combining Enable, Clamp, Shift, and Multiplier fields.
- **calculate_approx_params:** Calculate values of multiplication (M) and shift (s) for divider approximation.
- **send_padded_column:** Accepts an unsigned 8-bit array of size 7 as input and sends a column of 7 input pixels with automatically Zero-Padding to registers. This is because applying an image filter at the edge of the image requires an additional 0 value to prevent errors.
- **get_filter_output:** read output value as unsigned 8-bit from output register.
- **apply_hardware_filter:** main function that divides the image into sections and sequentially transmits them to the IP block, returning the output in an output array.

4. Verification & Testing Strategy

A two-step verification approach was used: Behavioral Simulation followed by In-System Verification.

Stage 1: VHDL Testbench (tb_filter_core.vhd) This verified the core logic in cycle-accurate simulation.

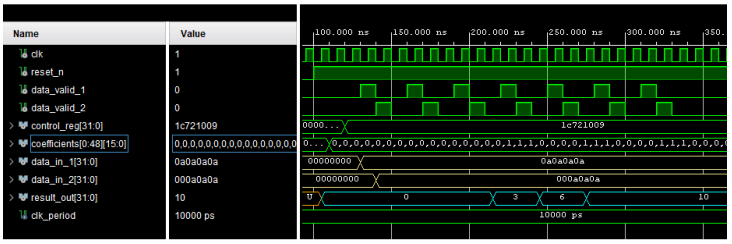
Gain Verification

10	10	10	10	10	10	10
10	10	10	10	10	10	10
10	10	10	10	10	10	10
10	10	10	10	10	10	10
10	10	10	10	10	10	10
10	10	10	10	10	10	10
10	10	10	10	10	10	10

INPUT

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Kernel



Result

- **Gain Verification:** A "Box Blur" 3x3 kernel (all 1s) was loaded and a flat field of value 10 was streamed. The simulation confirmed the expected sum (90) was correctly divided by 9 to output 0x0A (10).

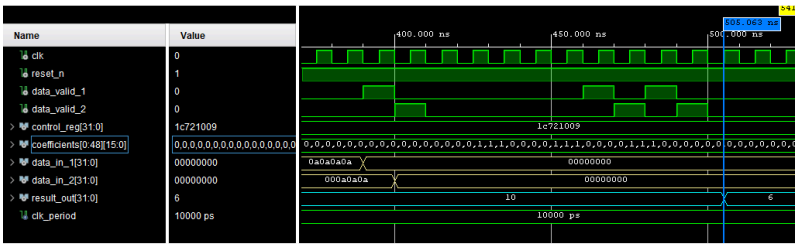
Timing Verification

10	10	10	10	0	0	0
10	10	10	10	0	0	0
10	10	10	10	0	0	0
10	10	10	10	0	0	0
10	10	10	10	0	0	0
10	10	10	10	0	0	0
10	10	10	10	0	0	0

INPUT

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

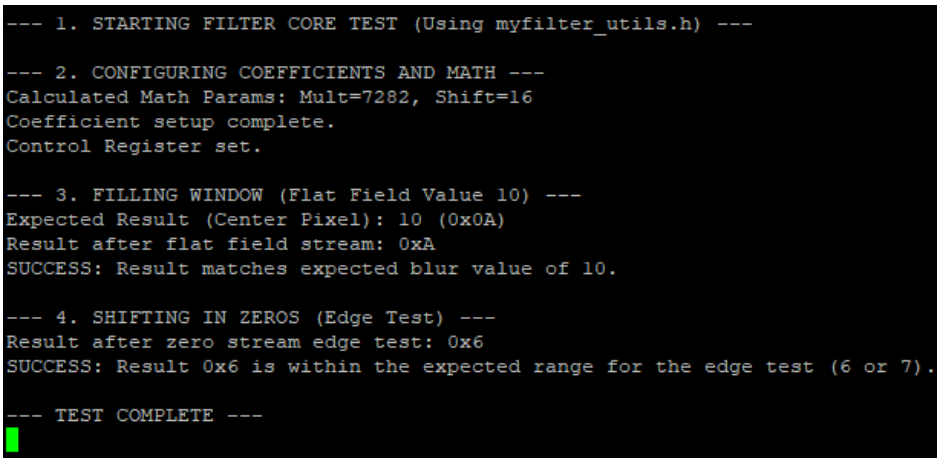
Kernel



Result

- **Timing Verification:** Three columns of 0s were streamed to simulate an edge. The output dropped to 0x06 exactly when expected (within the next clock), verifying the internal shift register timing.

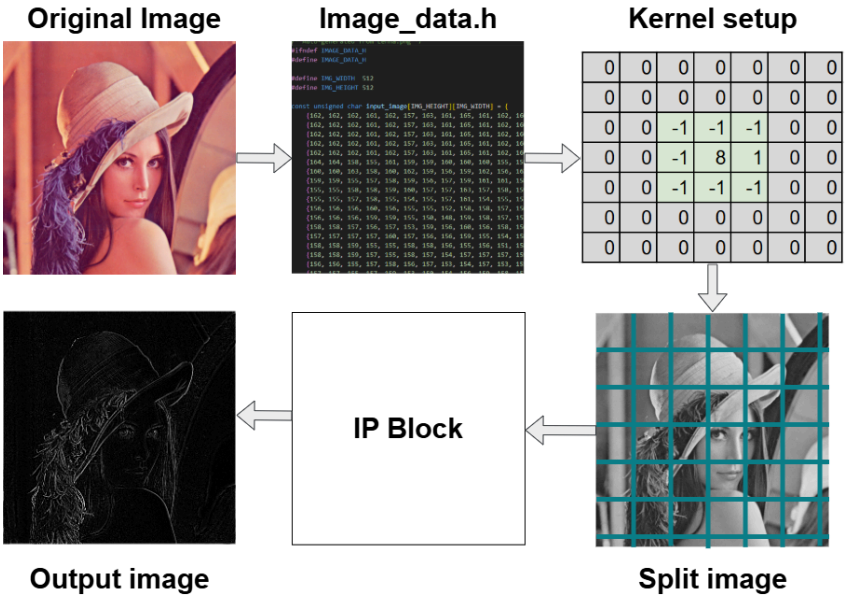
Stage 2: Hardware Validation (C-Based Dual Verification) The second stage involved performing the exact same verification steps as the testbench but executing them on the Zedboard hardware using C code.



- **Execution Consistency:** The C application was programmed to follow the testbench logic precisely. It initialized the AXI interface, loaded the same test kernels, and pushed identical pixel patterns into the “DATA_IN” registers.
- **System Synchronization:** The software was designed to monitor the AXI handshake signals, ensuring that data was only read once the hardware core had finished the convolution calculation.
- **Result Verification:** By running the same patterns through both the simulation and the C code, I was able to confirm that the hardware produced bit-exact results. This confirmed that the physical implementation matched the theoretical design.

5. Application Use Case

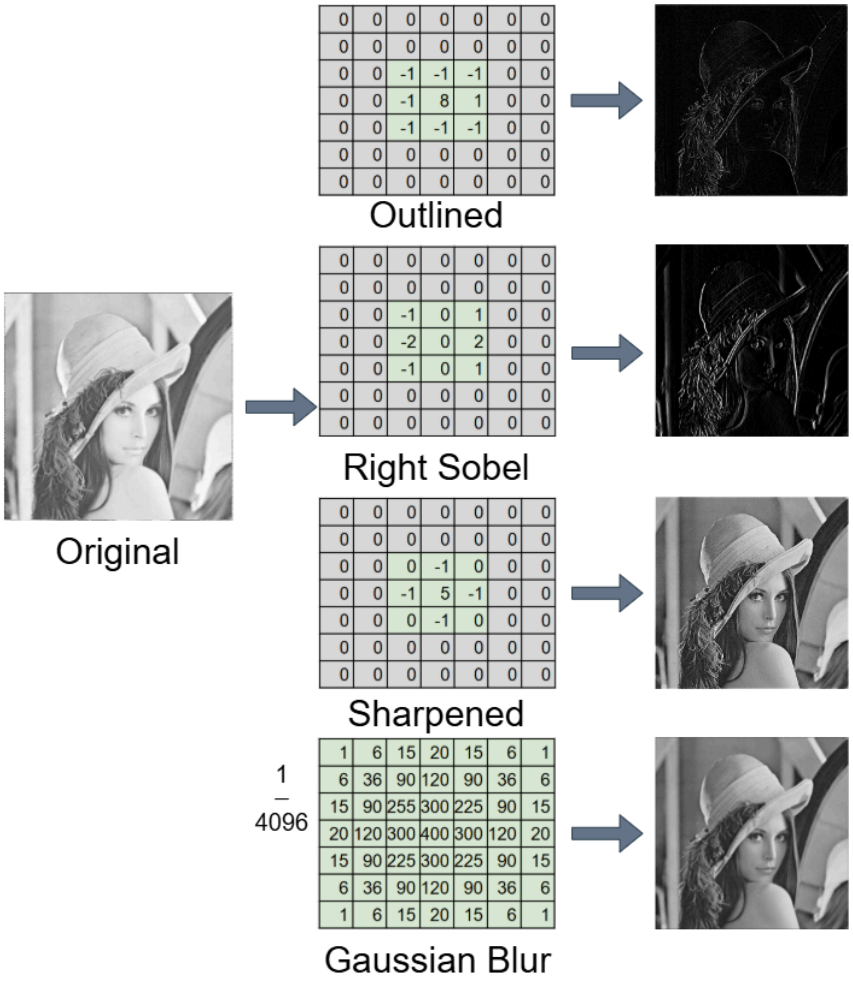
This part shows how to implement the hardware filter using a C-based workflow and the 512x512 "Lenna" test image. To make sure that data could be handled easily between the computer and the Zedboard hardware, the process was broken down into four basic steps.



Implementation Process The filtering task was achieved by combined Python scripting and C programming:

1. **Image Pre-processing**: Since the hardware requires raw pixel data, a Python script (image_to_header.py) was used to convert the original "Lenna.png" into a grayscale format. The script then generated a C header file (image_data.h) containing the image dimensions and a large array of pixel values (0-255). This allowed the image to be compiled directly into the Zedboard's memory.
2. **Hardware Initialization**: Within the C application, the software used “myfilter_utils.h” to define the initial state of the system. This included setting the 7x7 kernel coefficients and configuring the IP control registers (Enable, ABS, and Clamp). A critical step was the automated calculation of the multiplier and shift values to handle the filter's divisor, ensuring the brightness of the filtered image remained accurate.
3. **Hardware Filtering Execution**: The core processing was handled by the “apply_hardware_filter” function. This function automatically sliced the 512x512 image (can be any size) into manageable columns. It streamed these columns into the IP block using the column-updating technique discussed previously. The hardware performed the convolution in real-time, and the PS (Processing System) retrieved the results pixel-by-pixel then stored them in an output array.

4. **Data Export and Reconstruction**: After processing, each filtered pixel was sent from the Zedboard to the PC via a serial UART connection. A final Python script (serial_image.py) captured this serial data stream and reconstructed the individual pixels back into a viewable image file.



Result Summary By following this method, different filters were successfully applied. The output images confirmed that the hardware IP could process large data sets accurately. This workflow demonstrates a complete cycle of taking a standard image, processing it through custom FPGA hardware, and retrieving the final result for analysis.